Agenda: Week 3

1. Q&A
2. Dictionaries
    - What are they?
    - How to create them
    - How to work with them
    - How to iterate over them
    - The three paradigms of dict usage
    - How dicts are implemented behind the scenes
3. Files
    - Why we should think of files as data structures
    - How to read from files (in a few different ways)
    - A bit about writing to files, as well
    - The `with` construct, and how to use it (and why)

Dictionaries (`dict`)

We've seen in the last two weeks a few different data structures:

- Numbers (`int` and `float`)
- Text (`str`)
- Container sequences (`list` and `tuple`)

In the case of strings, lists, and tuples, the values are identified with an index that starts at 0 and goes up to the `len` - 1. This works, but it means that the index has no relationship with our data. A value could be at index 2, or 10, or 158.

But what if we have workers at our company with ID numbers? Then we'll (maybe) store a tuple about each worker at a different element in a list, but that element number and the ID number will have nothing to do with one another.

You can think of a dict as kind of like a list, except that you can decide what the index is (we call it a "key" in dicts) for each value. That is: You can use any number you want, not just 0, 1, 2, 3, etc. You can even use strings or tuples as your keys.

Better yet, searching and retrieving from a dict is extremely efficient and fast -- so if you're going to search for a value in a list, you might (sometimes, often?) be better off using a dict.

Dicts aren't only in Python

Other languages call our "dicts" by other names:

- Hash tables
- Hash maps
- Maps
- Hashes
- Key-value stores
- Name-value stores
- Associative arrays

The basic idea, no matter the name and no matter the language is:

- You are storing pairs of information
- The pair has a *key* and it has a *value*
- The key is what we use to store, retrieve, and search
- The value is ultimately what's interesting/important for us

To create a dict

In Python, we use `{}` for our dictionaries:

- `{}` by themselves are the "empty dict," similar to the empty string, empty list, and empty tuple
- Use `{}` with one or more pairs for a non-empty dict
- Each key is separated from its value with `:`, e.g., `'a':10`
- Each pair is separated from other pairs with `,`, e.g., `'a':10, 'b':20`

```
In [1]: d = {'a':10, 'b':20, 'c':30}  # this dict has 3 name-value pairs  -- don't t

In [2]: len(d)   # how many key-value pairs are in d?

Out[2]: 3

In [3]: d['a']  # retrieve the value associated with 'a'

Out[3]: 10

In [4]: d[a]   # this means: retrieve the value associated with the variable a's val
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[4], line 1
----> 1 d[a]   # this means: retrieve the value associated with the variable
a's value!

NameError: name 'a' is not defined
```

Rules for creating dicts

1. A dict can contain any number of key-value pairs. The minimum is zero (the empty dict, `{}`), and the maximum is determined by the memory in your computer.
2. Any immutable value in Python (i.e., basically numbers, strings, and tuples) can be a dict key. Normally, we use numbers and strings as keys. (This rule is slightly wrong, but not enough to cause trouble for you at this point.)
3. In any given dict, the keys must be unique
4. Every key has a value, and every value has a key.
5. There are NO RESTRICTIONS WHATSOEVER on values. They can be mutable. They can repeat. They can be big, nested, combinations of things.

In [5]:
```
d
```

Out[5]:
```
{'a': 10, 'b': 20, 'c': 30}
```

In [6]:
```
d['a']   # retrieve the value for key 'a'
```

Out[6]:
```
10
```

In [7]:
```
d['b']  # retrieve the value for key 'b'
```

Out[7]:
```
20
```

In [8]:
```
d['C']   # retrieve the value for key 'C'  (capital!)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[8], line 1
----> 1 d['C']   # retrieve the value for key 'C'  (capital!)

KeyError: 'C'
```

You have to be very careful with dict keys -- they only return a value if there is a 100% match. Capitalization, whitespace, punctuation... all of those are crucial.

In [10]:
```
k = 'c'

d[k]  # will this work? Yes, because this is turned into d['c'], which retur
```

Out[10]:
```
30
```

Where is this useful?

Answer: Absolutely everywhere.

- A dict whose keys are the names of the months of the year, and the values are the integers 1-12.
- A dict whose keys are the numbers of the months (1-2), and the values are the names.
- A dict whose keys are user IDs and whose values are usernames.
- A dict whose keys are filenames and whose values are data about each of those files.

What if a key doesn't exist?

We've already seen that retrieving a key that doesn't exist results in a `KeyError`. But we can prevent that from happening by first checking if a key is in a dict. We do this with `in`, the operator we've already seen for strings, lists, and tuples. Here, though, it only looks in the keys -- never in the values.

```
In [11]: d
```

```
Out[11]: {'a': 10, 'b': 20, 'c': 30}
```

```
In [12]: 'a' in d   # is 'a' a key in d?
```

```
Out[12]: True
```

```
In [13]: 'c' in d    # is 'c' a key in d?
```

```
Out[13]: True
```

```
In [14]: 30 in d   # is 30 a key in d?
```

```
Out[14]: False
```

Exercise: Restaurant

1. Define a dict, `menu`, which will contain restaurant menu items (strings) and their prices (integers).
2. Define `total` to be 0.
3. Ask the user (customer) repeatedly what they want to order from the menu.
    - If they enter an empty string, that shows they're done ordering. Print the total and be done.

- If they enter the name of something on the menu, then say how much it costs, how much the total is now, and print that on the screen.
- If they enter the name of something NOT on the menu, then apologize/scold and let them try again.

4. Print `total`.

Example:

```
Order: sandwich
sandwich is 15, total is 15
Order: tea
tea is 7, total is 22
Order: elephant
Sorry, we're fresh out of elephant today!
Order: [ENTER]
Total is 22.
```

In [15]:
```
menu = {'sandwich':15, 'tea':7, 'apple':1, 'cake':9}

print(menu)
```

```
{'sandwich': 15, 'tea': 7, 'apple': 1, 'cake': 9}
```

In [16]:
```
type(menu)
```

Out[16]:
```
dict
```

In [18]:
```
menu = {'sandwich':15, 'tea':7, 'apple':1, 'cake':9}
total = 0

while True:   # an infinite loop!
    order = input('Order: ').strip()

    if order == '':  # if we got the empty string,
        break

    if order in menu:          # check if the value of "order" is a key in t
        price = menu[order]
        total += price
        print(f'{order} costs {price}; total is now {total}')
    else:
        print(f'Sorry, we are fresh out of {order} today.')

print(f'In the end, your total was {total}.')
```

```
sandwich costs 15; total is now 15
tea costs 7; total is now 22
cake costs 9; total is now 31
tea costs 7; total is now 38
Sorry, we are fresh out of banana today.
In the end, your total was 38.
```

Are dicts mutable?

We've already discussed that strings and tuples are immutable -- they cannot be changed, once we have defined them. Lists, of course, are mutable -- we can modify existing values, add new values, and remove existing ones.

What about dictionaries?

They are *mutable*.

- We can replace an existing value
- We can add a new key-value pair
- We can remove an existing key-value pair

In [19]:
```
d = {'a':10, 'b':20, 'c':30}

# How do I replace a value in this dict?
# answer: just assign with []!

d['a'] = 999
```

In [20]:
```
d
```

Out[20]:
```
{'a': 999, 'b': 20, 'c': 30}
```

In [21]:
```
# How do I add a new key-value pair?
# Same way as we update a value -- just assign to the key

# If the key already exists, it's an update
# If the key doesn't yet exist, it's an addition to the dict

d['xyz'] = 987
d
```

Out[21]:
```
{'a': 999, 'b': 20, 'c': 30, 'xyz': 987}
```

In [22]:
```
# How do I remove a key-value pair?
# we invoke dict.pop() on the key.
# The key-value pair is removed *and* the value is returned to the caller.

d.pop('xyz')
```

Out[22]:
```
987
```

In [23]:
```
d
```

Out[23]:
```
{'a': 999, 'b': 20, 'c': 30}
```

Are keys kept in sorted order?

Sort of.

For a very long time, dicts were not stored in any predictable or guaranteed order. What came first vs. second when you iterated over it was a matter of a lot of luck.

In Python 3.6, they changed how dicts are implemented, and one side effect of that was that dicts are kept in *chronological* order. That is, the first key-value pair assigned is first, the second is second, etc.

In [24]:
```
d
```

Out[24]:
```
{'a': 999, 'b': 20, 'c': 30}
```

In [25]:
```
d.pop('a')
d.pop('c')
```

Out[25]:
```
30
```

In [26]:
```
d['c'] = 888
d['a'] = 30
```

In [27]:
```
d
```

Out[27]:
```
{'b': 20, 'c': 888, 'a': 30}
```

Exercise: Vowels, digits, and others (dict edition)

1. Define a dict, `counts`, with three keys -- `vowels`, `digits`, and `others`. All three values for these keys should be 0.
2. Ask the user to enter a string.
3. Go through the string, one character at a time:
   - If it's a vowel, add 1 to the value for `vowels`
   - If it's a digit, add 1 to the value for `digits`
   - If it's neither, add 1 to the value for `others`
4. Print `counts` in the end.

In [29]:
```
counts = {'vowels':0,
          'digits':0,
          'others':0}

text = input('Enter text: ').strip()
```

```
for one_character in text:
    if one_character in 'aeiou':   # if one_character is a vowel...
        counts['vowels'] += 1      # ... then add 1 to vowels
    elif one_character.isdigit():  # if one_character is a digit...
        counts['digits'] += 1      # ... then add 1 to digits
    else:
        counts['others'] += 1      # otherwise, just add to others

print(counts)
```

```
{'vowels': 2, 'digits': 3, 'others': 6}
```

Using `[]` vs. others

It's true that we use different kinds of parentheses to create data structures:

- `[]` to create a list
- `()` to create a tuple
- `{}` to create a dict

However, in all of these cases, as well as in strings, we use `[]` to retrieve an element. Why?

The answer is that when I say `x[i]` in Python, that is translated behind the scenes to what's known as a "magic method," one that we don't run ourselves, but that is run our behalf. In this case, the method is known as `__getitem__` ("dunder getitem"). So our invocation of `x[i]` becomes

```
x.__getitem__(i)
```

Any object that defines `__getitem__` then can use `[]`. In the case of these core data structures, they support `[]` for that reason.

Next up

1. Accumulating another way in dicts
2. Accumulating the unknown
3. Iterating over dicts

What can we store in a dict?

Yes, the keys in a dict are important -- they are the way that we get data in and out.

But because the values can be absolutely anything at all in Python, we can create complex data structures that combine the core ones. You can store integer and strings as a dict's values. You can also store lists. Or dicts. A dict of dicts is very useful -- the outer key is used to retrieve the record, and the inner key is used to retrieve a field in that record.

- If you have a list inside of a dict, just say `d['x'].append(1)`, and you've added 1 to the list that is stored for the `x` key.
- If you have a dict inside of a list, just say `d['x']['y'] += 5`, and you've added 5 to the dict key `y` within the dict key `x`.

Exercise: Vowels, digits, and others (dict-list edition)

1. Define a dict, `counts`, with three keys -- `vowels`, `digits`, and `others`. The values for all three keys should be `[]`, the empty list.
2. Ask the user to enter a string.
3. Go through that string, one character at a time.
    - If it's a vowel, append that character to the `vowels` list
    - If it's a digit, append the digit to the `digits` list
    - Otherwise, append the character to the `others` list
4. Print `counts` in the end. You will see a dict of lists.

In [30]:
```
counts = {'vowels':[],
          'digits':[],
          'others':[]}

text = input('Enter text: ').strip()

for one_character in text:
    if one_character in 'aeiou':                 # if one_character is a vow
        counts['vowels'].append(one_character)   # ... then add 1 to vowels
    elif one_character.isdigit():                # if one_character is a dig
        counts['digits'].append(one_character)   # ... then add 1 to digits
    else:
        counts['others'].append(one_character)   # otherwise, just add to ot

print(counts)
```

```
{'vowels': ['e', 'o'], 'digits': ['1', '2', '3'], 'others': ['h', 'l', 'l',
'!', '!', ' ']}
```

Three paradigms of dict usage

There are a lot of ways to use a dict. But in many cases, they fit into three different paradigms. We've already seen two:

1. Define a dict, and never change it. Treat it like a read-only database.
2. Define a dict with keys and initial values (0, empty list, etc). The keys won't change, and we won't add/remove keys from the dict. But the values will be changed, and will be updated to reflect what the program has seen/computed. It's a way to accumulate over time using one variable rather than a bunch.
3. Define an empty dict, and then count the unknown -- that is, we don't know what we'll get or see, but we do know that we want to count whatever comes our way. Creating a dict with all possible keys is just too time- or space-consuming.

In [34]:
```
# count characters in text
# I want to ask the user to enter some text, and count how many times each c

counts = {'a':0, 'b':0, 'c':0, 'd':0, 'e':0}

text = input('Enter text: ').strip()

for one_character in text:
    counts[one_character] += 1

print(counts)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[34], line 9
      5
      6 text = input('Enter text: ').strip()
      7
      8 for one_character in text:
----> 9     counts[one_character] += 1
     10
     11 print(counts)

KeyError: 'h'
```

Defining the dict with all possible characters as keys is just not realistic in this case. We'll do it in a different way:

1. The first time we see a character, we add a new key-value pair to our dict
2. The next time we see it, we add 1 to the value

In [37]:
```
# version 2 of our character counter

counts = {}

text = input('Enter text: ').strip()

for one_character in text:
    if one_character in counts:     # have we seen this character before?
        counts[one_character] += 1  # ... if so, add 1 to the count
    else:
        counts[one_character] = 1   # if it's new, then add a new key-value
```

```
print(counts)
```

```
{'h': 2, 'e': 6, 'l': 2, 'o': 3, ' ': 3, 'u': 1, 't': 2, 'r': 2, 'v': 1,
'y': 1, 'n': 1}
```

Exercise: Rainfall

The goal of this exercise is to keep track of the rainfall (in mm) in different cities over time. We don't know what cities we'll be measuring. We'll rely on the user to enter the name of a city, and then the amount of rain that fell there today. The dict keys will be strings (city names). The values will be integers (total mm rain in a city).

1. Define an empty dict, `rainfall`.
2. Ask the user to repeatedly enter the name of a city.
    - If the user enters an empty city name, break from the loop.
3. If you got a city name, then ask the user to enter mm rain.
    - Here, you can assume you'll get digits from the user
4. If you have seen data from this city before, add it to the existing number.
5. If you have not seen data from this city before, add a new key-value pair.
6. When the user finishes entering data, print the entire `rainfall` dict.

In [38]:
```
rainfall = {}

while True:
    city_name = input('City: ').strip()

    if city_name == '':
        break

    mm_rain = input('Rainfall: ').strip()
    mm_rain = int(mm_rain)   # big assumption -- that we will get only integ

    if city_name in rainfall:           # have we seen this city before?
        rainfall[city_name] += mm_rain  # ... if so, then just add mm_rain t
    else:
        rainfall[city_name] = mm_rain   # haven't seen this city before, so

print(rainfall)
```

```
{'a': 7, 'b': 4, 'c': 3}
```

Iteration and dicts

We've seen that we can iterate over a number of data structures with a `for` loop:

- Strings give us characters
- Iterating over a `range` gives us numbers

- Iterating over a list or tuple gives us the values

So... what happens if we iterate over a dict?

In [40]:
```
d = {'a':10, 'b':20, 'c':30}

for key in d:
    print(key)
```

```
a
b
c
```

In [41]:
```
# if I want to print all of the key-value pairs

for key in d:   # if I iterate over a dict, I get the keys, and the keys onl
    print(f'{key}: {d[key]}')
```

```
a: 10
b: 20
c: 30
```

In [42]:
```
# if we iterate over dict.items, we get key-value pairs (via a tuple and tup

for one_pair in d.items():
    print(one_pair)
```

```
('a', 10)
('b', 20)
('c', 30)
```

In [44]:
```
# my favorite way is with dict.items


for one_key, one_value in d.items():
    print(f'{one_key}: {one_value}')
```

```
a: 10
b: 20
c: 30
```

Next up

1. How dicts are implemented behind the scenes
2. Files - reading from them

You might want to download from https://files.lerner.co.il/exercise-files.zip .

How are dictionaries implemented?

The simplest way to understand a dict is sort of like a two-column table, like we saw in the Python Tutor display. The first column is keys and the second column is values. We

ask for a value based on a key, and we get it. We store a value based on a key, and it's stored.

But that makes it look like the key-value pairs are stored in order, or in something we can track easily.

The more complex story is that key-value pairs are stored in Python not in order as we saw, but rather based on the result of running the `hash` function on the key.

That is: When we store a key-value pair, we store it in a location based on the output from `hash(key)`.

What is `hash`? It's a function that returns an integer that *looks* random, but isn't. The idea is that this spreads the key-value pairs all over memory, and avoids the chance of a collision -- two different keys being stored in the same place.

Why go through all of this effort to store our key-value pairs?

Answer is: Super duper fast lookup and retrieval.

If you ask `'a' in d`, Python runs `hash('a')`, jumps to that location in memory, and can tell you right away if it's there.

The speed with which we can search in a dict's keys, or retrieve a value from a key, doesn't depend on the size of the dict. So whether it has 5 pairs or 5m pairs, Python will still find items in a dict very quickly.

Python loves dicts so much that it uses them to implement many of its features:

- Variables are strings -- and Python stores them in a dict, behind the scenes, with its values.
- Attributes, the names that come after `.`, are stored in a dict

So what problems are there with dicts?

In [45]:
```
# 1. Only immutable values can be keys. (Not totally true, but true enough.)

mylist = [10, 20, 30, 40, 50]

d = {mylist:123}
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[45], line 5
      1 # 1. Only immutable values can be keys. (Not totally true, but true
enough.)
      2
      3 mylist = [10, 20, 30, 40, 50]
      4
----> 5 d = {mylist:123}

TypeError: cannot use 'list' as a dict key (unhashable type: 'list')
```

In [46]:
```
hash(mylist)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[46], line 1
----> 1 hash(mylist)

TypeError: unhashable type: 'list'
```

Files

We use files all of the time!

- PDF
- Excel
- PowerPoint
- Python files
- JPEGs, MP4

What is a file?

You can think of a file as a stored version of data structures from your computer's memory. The data structure only lasts so long as the computer is on, and only on one computer. To share it with others, or have a copy around when the user's computer restarts, is useful and necessary.

What we can do is take the data structures in memory, and dump them directly to a special file. And then, we can read them from the special file back into memory, and it's as if we never left.

Have you seen this before? Yes -- with Exel, Word, PowerPoint, etc.

Python can work with files -- both for writing and reading. However, we're going to stick with text files in this course -- they're common and also easy to work with.

In real life, you probably will work with other formats.

We want to work with a file. How can we connect to it?

We can't, at least not direclty -- it would violate all sorts of security on your computer and operating system. Rather, you have to go through the OS.

1. You `open` a file, telling the OS you want to read from it
2. Then you can read its contents into memory
3. Then you can (if appropriate) use the content with your program

Whatever we read from a file comes as a text string.

In [47]:
```
filename = 'passwd.txt'

f = open(filename)  # open the file - -what we get back is a "file object,"
print(f.read())   # this returns the entirety of the file as one string
```

```
nobody:*:-2:-2:Unprivileged User:/var/empty:/usr/bin/false
root:*:0:0:System Administrator:/var/root:/bin/sh
daemon:*:1:1:System Services:/var/root:/usr/bin/false
_uucp:*:4:4:Unix to Unix Copy Protocol:/var/spool/uucp:/usr/sbin/uucico
_taskgated:*:13:13:Task Gate Daemon:/var/empty:/usr/bin/false
_networkd:*:24:24:Network Services:/var/networkd:/usr/bin/false
_installassistant:*:25:25:Install Assistant:/var/empty:/usr/bin/false
_lp:*:26:26:Printing Services:/var/spool/cups:/usr/bin/false
_postfix:*:27:27:Postfix Mail Server:/var/spool/postfix:/usr/bin/false
_scsd:*:31:31:Service Configuration Service:/var/empty:/usr/bin/false
_ces:*:32:32:Certificate Enrollment Service:/var/empty:/usr/bin/false
_appstore:*:33:33:Mac App Store Service:/var/db/appstore:/usr/bin/false
_mcxalr:*:54:54:MCX AppLaunch:/var/empty:/usr/bin/false
_appleevents:*:55:55:AppleEvents Daemon:/var/empty:/usr/bin/false
_geod:*:56:56:Geo Services Daemon:/var/db/geod:/usr/bin/false
_devdocs:*:59:59:Developer Documentation:/var/empty:/usr/bin/false
_sandbox:*:60:60:Seatbelt:/var/empty:/usr/bin/false
_mdnsresponder:*:65:65:mDNSResponder:/var/empty:/usr/bin/false
_ard:*:67:67:Apple Remote Desktop:/var/empty:/usr/bin/false
_www:*:70:70:World Wide Web Server:/Library/WebServer:/usr/bin/false
_eppc:*:71:71:Apple Events User:/var/empty:/usr/bin/false
_cvs:*:72:72:CVS Server:/var/empty:/usr/bin/false
_svn:*:73:73:SVN Server:/var/empty:/usr/bin/false
_mysql:*:74:74:MySQL Server:/var/empty:/usr/bin/false
_sshd:*:75:75:sshd Privilege separation:/var/empty:/usr/bin/false
_qtss:*:76:76:QuickTime Streaming Server:/var/empty:/usr/bin/false
_cyrus:*:77:6:Cyrus Administrator:/var/imap:/usr/bin/false
_mailman:*:78:78:Mailman List Server:/var/empty:/usr/bin/false
_appserver:*:79:79:Application Server:/var/empty:/usr/bin/false
_clamav:*:82:82:ClamAV Daemon:/var/virusmails:/usr/bin/false
_amavisd:*:83:83:AMaViS Daemon:/var/virusmails:/usr/bin/false
_jabber:*:84:84:Jabber XMPP Server:/var/empty:/usr/bin/false
_appowner:*:87:87:Application Owner:/var/empty:/usr/bin/false
_windowserver:*:88:88:WindowServer:/var/empty:/usr/bin/false
_spotlight:*:89:89:Spotlight:/var/empty:/usr/bin/false
_tokend:*:91:91:Token Daemon:/var/empty:/usr/bin/false
_securityagent:*:92:92:SecurityAgent:/var/db/securityagent:/usr/bin/false
_calendar:*:93:93:Calendar:/var/empty:/usr/bin/false
_teamsserver:*:94:94:TeamsServer:/var/teamsserver:/usr/bin/false
_update_sharing:*:95:-2:Update Sharing:/var/empty:/usr/bin/false
_installer:*:96:-2:Installer:/var/empty:/usr/bin/false
_atsserver:*:97:97:ATS Server:/var/empty:/usr/bin/false
_ftp:*:98:-2:FTP Daemon:/var/empty:/usr/bin/false
_unknown:*:99:99:Unknown User:/var/empty:/usr/bin/false
_softwareupdate:*:200:200:Software Update Service:/var/db/softwareupdate:/us
r/bin/false
_coreaudiod:*:202:202:Core Audio Daemon:/var/empty:/usr/bin/false
_screensaver:*:203:203:Screensaver:/var/empty:/usr/bin/false
_locationd:*:205:205:Location Daemon:/var/db/locationd:/usr/bin/false
_trustevaluationagent:*:208:208:Trust Evaluation Agent:/var/empty:/usr/bin/f
alse
_timezone:*:210:210:AutoTimeZoneDaemon:/var/empty:/usr/bin/false
_lda:*:211:211:Local Delivery Agent:/var/empty:/usr/bin/false
_cvmsroot:*:212:212:CVMS Root:/var/empty:/usr/bin/false
_usbmuxd:*:213:213:iPhone OS Device Helper:/var/db/lockdown:/usr/bin/false
_dovecot:*:214:6:Dovecot Administrator:/var/empty:/usr/bin/false
```

```
_dpaudio:*:215:215:DP Audio:/var/empty:/usr/bin/false
_postgres:*:216:216:PostgreSQL Server:/var/empty:/usr/bin/false
_krbtgt:*:217:-2:Kerberos Ticket Granting Ticket:/var/empty:/usr/bin/false
_kadmin_admin:*:218:-2:Kerberos Admin Service:/var/empty:/usr/bin/false
_kadmin_changepw:*:219:-2:Kerberos Change Password Service:/var/empty:/usr/b
in/false
_devicemgr:*:220:220:Device Management Server:/var/empty:/usr/bin/false
_webauthserver:*:221:221:Web Auth Server:/var/empty:/usr/bin/false
_netbios:*:222:222:NetBIOS:/var/empty:/usr/bin/false
_warmd:*:224:224:Warm Daemon:/var/empty:/usr/bin/false
_dovenull:*:227:227:Dovecot Authentication:/var/empty:/usr/bin/false
_netstatistics:*:228:228:Network Statistics Daemon:/var/empty:/usr/bin/false
_avbdeviced:*:229:-2:Ethernet AVB Device Daemon:/var/empty:/usr/bin/false
_krb_krbtgt:*:230:-2:Open Directory Kerberos Ticket Granting Ticket:/var/emp
ty:/usr/bin/false
_krb_kadmin:*:231:-2:Open Directory Kerberos Admin Service:/var/empty:/usr/b
in/false
_krb_changepw:*:232:-2:Open Directory Kerberos Change Password Service:/var/
empty:/usr/bin/false
_krb_kerberos:*:233:-2:Open Directory Kerberos:/var/empty:/usr/bin/false
_krb_anonymous:*:234:-2:Open Directory Kerberos Anonymous:/var/empty:/usr/bi
n/false
_assetcache:*:235:235:Asset Cache Service:/var/empty:/usr/bin/false
_coremediaiod:*:236:236:Core Media IO Daemon:/var/empty:/usr/bin/false
_launchservicesd:*:239:239:_launchservicesd:/var/empty:/usr/bin/false
_iconservices:*:240:240:IconServices:/var/empty:/usr/bin/false
_distnote:*:241:241:DistNote:/var/empty:/usr/bin/false
_nsurlsessiond:*:242:242:NSURLSession Daemon:/var/db/nsurlsessiond:/usr/bin/
false
_displaypolicyd:*:244:244:Display Policy Daemon:/var/empty:/usr/bin/false
_astris:*:245:245:Astris Services:/var/db/astris:/usr/bin/false
_krbfast:*:246:-2:Kerberos FAST Account:/var/empty:/usr/bin/false
_gamecontrollerd:*:247:247:Game Controller Daemon:/var/empty:/usr/bin/false
_mbsetupuser:*:248:248:Setup User:/var/setup:/bin/bash
_ondemand:*:249:249:On Demand Resource Daemon:/var/db/ondemand:/usr/bin/fals
e
_xserverdocs:*:251:251:macOS Server Documents Service:/var/empty:/usr/bin/fa
lse
_wwwproxy:*:252:252:WWW Proxy:/var/empty:/usr/bin/false
_mobileasset:*:253:253:MobileAsset User:/var/ma:/usr/bin/false
_findmydevice:*:254:254:Find My Device Daemon:/var/db/findmydevice:/usr/bin/
false
_datadetectors:*:257:257:DataDetectors:/var/db/datadetectors:/usr/bin/false
_captiveagent:*:258:258:captiveagent:/var/empty:/usr/bin/false
_ctkd:*:259:259:ctkd Account:/var/db/ctkd:/usr/bin/false
_applepay:*:260:260:applepay Account:/var/db/applepay:/usr/bin/false
_hidd:*:261:261:HID Service User:/var/db/hidd:/usr/bin/false
_cmiodalassistants:*:262:262:CoreMedia IO Assistants User:/var/db/cmiodalass
istants:/usr/bin/false
_analyticsd:*:263:263:Analytics Daemon:/var/db/analyticsd:/usr/bin/false
_fpsd:*:265:265:FPS Daemon:/var/db/fpsd:/usr/bin/false
_timed:*:266:266:Time Sync Daemon:/var/db/timed:/usr/bin/false
_nearbyd:*:268:268:Proximity and Ranging Daemon:/var/db/nearbyd:/usr/bin/fal
se
_reportmemoryexception:*:269:269:ReportMemoryException:/var/db/reportmemorye
xception:/usr/bin/false
```

```
_driverkit:*:270:270:DriverKit:/var/empty:/usr/bin/false
_diskimagesiod:*:271:271:DiskImages IO Daemon:/var/db/diskimagesiod:/usr/bi
n/false
_logd:*:272:272:Log Daemon:/var/db/diagnostics:/usr/bin/false
_appinstalld:*:273:273:App Install Daemon:/var/db/appinstalld:/usr/bin/false
_installcoordinationd:*:274:274:Install Coordination Daemon:/var/db/installc
oordinationd:/usr/bin/false
_demod:*:275:275:Demo Daemon:/var/empty:/usr/bin/false
_rmd:*:277:277:Remote Management Daemon:/var/db/rmd:/usr/bin/false
_accessoryupdater:*:278:278:Accessory Update Daemon:/var/db/accessoryupdate
r:/usr/bin/false
_knowledgegraphd:*:279:279:Knowledge Graph Daemon:/var/db/knowledgegraphd:/u
sr/bin/false
_coreml:*:280:280:CoreML Services:/var/db/coreml:/usr/bin/false
_sntpd:*:281:281:SNTP Server Daemon:/var/empty:/usr/bin/false
_trustd:*:282:282:trustd:/var/empty:/usr/bin/false
_mmaintenanced:*:283:283:mmaintenanced:/var/db/mmaintenanced:/usr/bin/false
_darwindaemon:*:284:284:Darwin Daemon:/var/db/darwindaemon:/usr/bin/false
_notification_proxy:*:285:285:Notification Proxy:/var/empty:/usr/bin/false
_avphidbridge:*:288:288:Apple Virtual Platform HID Bridge:/var/empty:/usr/bi
n/false
_biome:*:289:289:Biome:/var/db/biome:/usr/bin/false
_backgroundassets:*:291:291:Background Assets Service:/var/empty:/usr/bin/fa
lse
_mobilegestalthelper:*:293:293:MobileGestaltHelper:/var/empty:/usr/bin/false
_audiomxd:*:294:294:Audio and MediaExperience Daemon:/var/db/audiomxd:/usr/b
in/false
_terminusd:*:295:295:Terminus:/var/db/terminus:/usr/bin/false
_neuralengine:*:296:296:AppleNeuralEngine:/var/db/neuralengine:/usr/bin/fals
e
_eligibilityd:*:297:297:OS Eligibility Daemon:/var/db/eligibilityd:/usr/bin/
false
_systemstatusd:*:298:298:SystemStatus Services:/var/empty:/usr/bin/false
_aonsensed:*:300:300:Always On Sense Daemon:/var/db/aonsensed:/usr/bin/false
_modelmanagerd:*:301:301:Model Manager:/var/db/modelmanagerd:/usr/bin/false
_reportsystemmemory:*:302:302:ReportSystemMemory:/var/empty:/usr/bin/false
_swtransparencyd:*:303:303:Software Transparency Services:/var/db/swtranspar
encyd:/usr/bin/false
_naturallanguaged:*:304:304:Natural Language Services:/var/db/com.apple.natu
rallanguaged:/usr/bin/false
_spinandd:*:305:305:SPINAND Daemon:/var/empty:/usr/bin/false
_corespeechd:*:306:306:CoreSpeech Services:/var/empty:/usr/bin/false
_diagnosticservicesd:*:307:307:Diagnostic Services:/var/empty:/usr/bin/false
_mds_stores:*:308:308:Spotlight File Metadata Index Daemon:/var/empty:/usr/b
in/false
_oahd:*:441:441:OAH Daemon:/var/empty:/usr/bin/false
```

Don't use `read` to read from a file!

When you invoke `read`, you get back a string. What happens if you open a 100 TB file?

There is a better way to read from a file, and it involves the `for` loop:

In [49]:

```
filename = 'passwd.txt'

f = open(filename)

for one_line in f:    # by definition, looping over f returns a line
    print(one_line.strip())
```

```
nobody:*:-2:-2:Unprivileged User:/var/empty:/usr/bin/false
root:*:0:0:System Administrator:/var/root:/bin/sh
daemon:*:1:1:System Services:/var/root:/usr/bin/false
_uucp:*:4:4:Unix to Unix Copy Protocol:/var/spool/uucp:/usr/sbin/uucico
_taskgated:*:13:13:Task Gate Daemon:/var/empty:/usr/bin/false
_networkd:*:24:24:Network Services:/var/networkd:/usr/bin/false
_installassistant:*:25:25:Install Assistant:/var/empty:/usr/bin/false
_lp:*:26:26:Printing Services:/var/spool/cups:/usr/bin/false
_postfix:*:27:27:Postfix Mail Server:/var/spool/postfix:/usr/bin/false
_scsd:*:31:31:Service Configuration Service:/var/empty:/usr/bin/false
_ces:*:32:32:Certificate Enrollment Service:/var/empty:/usr/bin/false
_appstore:*:33:33:Mac App Store Service:/var/db/appstore:/usr/bin/false
_mcxalr:*:54:54:MCX AppLaunch:/var/empty:/usr/bin/false
_appleevents:*:55:55:AppleEvents Daemon:/var/empty:/usr/bin/false
_geod:*:56:56:Geo Services Daemon:/var/db/geod:/usr/bin/false
_devdocs:*:59:59:Developer Documentation:/var/empty:/usr/bin/false
_sandbox:*:60:60:Seatbelt:/var/empty:/usr/bin/false
_mdnsresponder:*:65:65:mDNSResponder:/var/empty:/usr/bin/false
_ard:*:67:67:Apple Remote Desktop:/var/empty:/usr/bin/false
_www:*:70:70:World Wide Web Server:/Library/WebServer:/usr/bin/false
_eppc:*:71:71:Apple Events User:/var/empty:/usr/bin/false
_cvs:*:72:72:CVS Server:/var/empty:/usr/bin/false
_svn:*:73:73:SVN Server:/var/empty:/usr/bin/false
_mysql:*:74:74:MySQL Server:/var/empty:/usr/bin/false
_sshd:*:75:75:sshd Privilege separation:/var/empty:/usr/bin/false
_qtss:*:76:76:QuickTime Streaming Server:/var/empty:/usr/bin/false
_cyrus:*:77:6:Cyrus Administrator:/var/imap:/usr/bin/false
_mailman:*:78:78:Mailman List Server:/var/empty:/usr/bin/false
_appserver:*:79:79:Application Server:/var/empty:/usr/bin/false
_clamav:*:82:82:ClamAV Daemon:/var/virusmails:/usr/bin/false
_amavisd:*:83:83:AMaViS Daemon:/var/virusmails:/usr/bin/false
_jabber:*:84:84:Jabber XMPP Server:/var/empty:/usr/bin/false
_appowner:*:87:87:Application Owner:/var/empty:/usr/bin/false
_windowserver:*:88:88:WindowServer:/var/empty:/usr/bin/false
_spotlight:*:89:89:Spotlight:/var/empty:/usr/bin/false
_tokend:*:91:91:Token Daemon:/var/empty:/usr/bin/false
_securityagent:*:92:92:SecurityAgent:/var/db/securityagent:/usr/bin/false
_calendar:*:93:93:Calendar:/var/empty:/usr/bin/false
_teamsserver:*:94:94:TeamsServer:/var/teamsserver:/usr/bin/false
_update_sharing:*:95:-2:Update Sharing:/var/empty:/usr/bin/false
_installer:*:96:-2:Installer:/var/empty:/usr/bin/false
_atsserver:*:97:97:ATS Server:/var/empty:/usr/bin/false
_ftp:*:98:-2:FTP Daemon:/var/empty:/usr/bin/false
_unknown:*:99:99:Unknown User:/var/empty:/usr/bin/false
_softwareupdate:*:200:200:Software Update Service:/var/db/softwareupdate:/us
r/bin/false
_coreaudiod:*:202:202:Core Audio Daemon:/var/empty:/usr/bin/false
_screensaver:*:203:203:Screensaver:/var/empty:/usr/bin/false
_locationd:*:205:205:Location Daemon:/var/db/locationd:/usr/bin/false
_trustevaluationagent:*:208:208:Trust Evaluation Agent:/var/empty:/usr/bin/f
alse
_timezone:*:210:210:AutoTimeZoneDaemon:/var/empty:/usr/bin/false
_lda:*:211:211:Local Delivery Agent:/var/empty:/usr/bin/false
_cvmsroot:*:212:212:CVMS Root:/var/empty:/usr/bin/false
_usbmuxd:*:213:213:iPhone OS Device Helper:/var/db/lockdown:/usr/bin/false
_dovecot:*:214:6:Dovecot Administrator:/var/empty:/usr/bin/false
```

```
_dpaudio:*:215:215:DP Audio:/var/empty:/usr/bin/false
_postgres:*:216:216:PostgreSQL Server:/var/empty:/usr/bin/false
_krbtgt:*:217:-2:Kerberos Ticket Granting Ticket:/var/empty:/usr/bin/false
_kadmin_admin:*:218:-2:Kerberos Admin Service:/var/empty:/usr/bin/false
_kadmin_changepw:*:219:-2:Kerberos Change Password Service:/var/empty:/usr/b
in/false
_devicemgr:*:220:220:Device Management Server:/var/empty:/usr/bin/false
_webauthserver:*:221:221:Web Auth Server:/var/empty:/usr/bin/false
_netbios:*:222:222:NetBIOS:/var/empty:/usr/bin/false
_warmd:*:224:224:Warm Daemon:/var/empty:/usr/bin/false
_dovenull:*:227:227:Dovecot Authentication:/var/empty:/usr/bin/false
_netstatistics:*:228:228:Network Statistics Daemon:/var/empty:/usr/bin/false
_avbdeviced:*:229:-2:Ethernet AVB Device Daemon:/var/empty:/usr/bin/false
_krb_krbtgt:*:230:-2:Open Directory Kerberos Ticket Granting Ticket:/var/emp
ty:/usr/bin/false
_krb_kadmin:*:231:-2:Open Directory Kerberos Admin Service:/var/empty:/usr/b
in/false
_krb_changepw:*:232:-2:Open Directory Kerberos Change Password Service:/var/
empty:/usr/bin/false
_krb_kerberos:*:233:-2:Open Directory Kerberos:/var/empty:/usr/bin/false
_krb_anonymous:*:234:-2:Open Directory Kerberos Anonymous:/var/empty:/usr/bi
n/false
_assetcache:*:235:235:Asset Cache Service:/var/empty:/usr/bin/false
_coremediaiod:*:236:236:Core Media IO Daemon:/var/empty:/usr/bin/false
_launchservicesd:*:239:239:_launchservicesd:/var/empty:/usr/bin/false
_iconservices:*:240:240:IconServices:/var/empty:/usr/bin/false
_distnote:*:241:241:DistNote:/var/empty:/usr/bin/false
_nsurlsessiond:*:242:242:NSURLSession Daemon:/var/db/nsurlsessiond:/usr/bin/
false
_displaypolicyd:*:244:244:Display Policy Daemon:/var/empty:/usr/bin/false
_astris:*:245:245:Astris Services:/var/db/astris:/usr/bin/false
_krbfast:*:246:-2:Kerberos FAST Account:/var/empty:/usr/bin/false
_gamecontrollerd:*:247:247:Game Controller Daemon:/var/empty:/usr/bin/false
_mbsetupuser:*:248:248:Setup User:/var/setup:/bin/bash
_ondemand:*:249:249:On Demand Resource Daemon:/var/db/ondemand:/usr/bin/fals
e
_xserverdocs:*:251:251:macOS Server Documents Service:/var/empty:/usr/bin/fa
lse
_wwwproxy:*:252:252:WWW Proxy:/var/empty:/usr/bin/false
_mobileasset:*:253:253:MobileAsset User:/var/ma:/usr/bin/false
_findmydevice:*:254:254:Find My Device Daemon:/var/db/findmydevice:/usr/bin/
false
_datadetectors:*:257:257:DataDetectors:/var/db/datadetectors:/usr/bin/false
_captiveagent:*:258:258:captiveagent:/var/empty:/usr/bin/false
_ctkd:*:259:259:ctkd Account:/var/db/ctkd:/usr/bin/false
_applepay:*:260:260:applepay Account:/var/db/applepay:/usr/bin/false
_hidd:*:261:261:HID Service User:/var/db/hidd:/usr/bin/false
_cmiodalassistants:*:262:262:CoreMedia IO Assistants User:/var/db/cmiodalass
istants:/usr/bin/false
_analyticsd:*:263:263:Analytics Daemon:/var/db/analyticsd:/usr/bin/false
_fpsd:*:265:265:FPS Daemon:/var/db/fpsd:/usr/bin/false
_timed:*:266:266:Time Sync Daemon:/var/db/timed:/usr/bin/false
_nearbyd:*:268:268:Proximity and Ranging Daemon:/var/db/nearbyd:/usr/bin/fal
se
_reportmemoryexception:*:269:269:ReportMemoryException:/var/db/reportmemorye
xception:/usr/bin/false
```

```
_driverkit:*:270:270:DriverKit:/var/empty:/usr/bin/false
_diskimagesiod:*:271:271:DiskImages IO Daemon:/var/db/diskimagesiod:/usr/bi
n/false
_logd:*:272:272:Log Daemon:/var/db/diagnostics:/usr/bin/false
_appinstalld:*:273:273:App Install Daemon:/var/db/appinstalld:/usr/bin/false
_installcoordinationd:*:274:274:Install Coordination Daemon:/var/db/installc
oordinationd:/usr/bin/false
_demod:*:275:275:Demo Daemon:/var/empty:/usr/bin/false
_rmd:*:277:277:Remote Management Daemon:/var/db/rmd:/usr/bin/false
_accessoryupdater:*:278:278:Accessory Update Daemon:/var/db/accessoryupdate
r:/usr/bin/false
_knowledgegraphd:*:279:279:Knowledge Graph Daemon:/var/db/knowledgegraphd:/u
sr/bin/false
_coreml:*:280:280:CoreML Services:/var/db/coreml:/usr/bin/false
_sntpd:*:281:281:SNTP Server Daemon:/var/empty:/usr/bin/false
_trustd:*:282:282:trustd:/var/empty:/usr/bin/false
_mmaintenanced:*:283:283:mmaintenanced:/var/db/mmaintenanced:/usr/bin/false
_darwindaemon:*:284:284:Darwin Daemon:/var/db/darwindaemon:/usr/bin/false
_notification_proxy:*:285:285:Notification Proxy:/var/empty:/usr/bin/false
_avphidbridge:*:288:288:Apple Virtual Platform HID Bridge:/var/empty:/usr/bi
n/false
_biome:*:289:289:Biome:/var/db/biome:/usr/bin/false
_backgroundassets:*:291:291:Background Assets Service:/var/empty:/usr/bin/fa
lse
_mobilegestalthelper:*:293:293:MobileGestaltHelper:/var/empty:/usr/bin/false
_audiomxd:*:294:294:Audio and MediaExperience Daemon:/var/db/audiomxd:/usr/b
in/false
_terminusd:*:295:295:Terminus:/var/db/terminus:/usr/bin/false
_neuralengine:*:296:296:AppleNeuralEngine:/var/db/neuralengine:/usr/bin/fals
e
_eligibilityd:*:297:297:OS Eligibility Daemon:/var/db/eligibilityd:/usr/bin/
false
_systemstatusd:*:298:298:SystemStatus Services:/var/empty:/usr/bin/false
_aonsensed:*:300:300:Always On Sense Daemon:/var/db/aonsensed:/usr/bin/false
_modelmanagerd:*:301:301:Model Manager:/var/db/modelmanagerd:/usr/bin/false
_reportsystemmemory:*:302:302:ReportSystemMemory:/var/empty:/usr/bin/false
_swtransparencyd:*:303:303:Software Transparency Services:/var/db/swtranspar
encyd:/usr/bin/false
_naturallanguaged:*:304:304:Natural Language Services:/var/db/com.apple.natu
rallanguaged:/usr/bin/false
_spinandd:*:305:305:SPINAND Daemon:/var/empty:/usr/bin/false
_corespeechd:*:306:306:CoreSpeech Services:/var/empty:/usr/bin/false
_diagnosticservicesd:*:307:307:Diagnostic Services:/var/empty:/usr/bin/false
_mds_stores:*:308:308:Spotlight File Metadata Index Daemon:/var/empty:/usr/b
in/false
_oahd:*:441:441:OAH Daemon:/var/empty:/usr/bin/false
```

Exercise: Print usernames

1. Open the file `passwd.txt`, and iterate over it line by line.
2. Print, for each line, the username at the start. (That's the first item of the line, ending with a `:`.

In [53]:

```
f = open('passwd.txt')

for one_line in f:
    print(one_line.split(':')[0])   # print the usernames -- split the line

f.close()  #
```

nobody
root
daemon
_uucp
_taskgated
_networkd
_installassistant
_lp
_postfix
_scsd
_ces
_appstore
_mcxalr
_appleevents
_geod
_devdocs
_sandbox
_mdnsresponder
_ard
_www
_eppc
_cvs
_svn
_mysql
_sshd
_qtss
_cyrus
_mailman
_appserver
_clamav
_amavisd
_jabber
_appowner
_windowserver
_spotlight
_tokend
_securityagent
_calendar
_teamsserver
_update_sharing
_installer
_atsserver
_ftp
_unknown
_softwareupdate
_coreaudiod
_screensaver
_locationd
_trustevaluationagent
_timezone
_lda
_cvmsroot
_usbmuxd
_dovecot
_dpaudio
_postgres

_krbtgt
_kadmin_admin
_kadmin_changepw
_devicemgr
_webauthserver
_netbios
_warmd
_dovenull
_netstatistics
_avbdeviced
_krb_krbtgt
_krb_kadmin
_krb_changepw
_krb_kerberos
_krb_anonymous
_assetcache
_coremediaiod
_launchservicesd
_iconservices
_distnote
_nsurlsessiond
_displaypolicyd
_astris
_krbfast
_gamecontrollerd
_mbsetupuser
_ondemand
_xserverdocs
_wwwproxy
_mobileasset
_findmydevice
_datadetectors
_captiveagent
_ctkd
_applepay
_hidd
_cmiodalassistants
_analyticsd
_fpsd
_timed
_nearbyd
_reportmemoryexception
_driverkit
_diskimagesiod
_logd
_appinstalld
_installcoordinationd
_demod
_rmd
_accessoryupdater
_knowledgegraphd
_coreml
_sntpd
_trustd
_mmaintenanced
_darwindaemon

```
_notification_proxy
_avphidbridge
_biome
_backgroundassets
_mobilegestalthelper
_audiomxd
_terminusd
_neuralengine
_eligibilityd
_systemstatusd
_aonsensed
_modelmanagerd
_reportsystemmemory
_swtransparencyd
_naturallanguaged
_spinandd
_corespeechd
_diagnosticservicesd
_mds_stores
_oahd
```

Exercise: Counting IP addresses

The zipfile contains a file called `mini-access-log.txt`, a very very short Apache logfile. The first part of each line is an IP address.

1. Go through the file, one line at a time, grabbing the IP address and printing it out.
2. Then once you can do that, create a `counts` empty dict. Go through each IP address, and count how many times it appears in the dict.

In [56]:
```
f = open('mini-access-log.txt')

for one_line in f:
    ip_address = one_line.split()[0]
    print(ip_address)

f.close()
```

67.218.116.165
66.249.71.65
65.55.106.183
65.55.106.183
66.249.71.65
66.249.71.65
66.249.65.12
66.249.65.12
66.249.65.12
66.249.65.12
66.249.65.12
65.55.106.131
65.55.106.131
66.249.65.12
66.249.65.12
66.249.65.12
66.249.65.12
66.249.65.12
66.249.65.12
66.249.65.12
65.55.106.186
65.55.106.186
66.249.65.12
66.249.65.12
66.249.65.12
74.52.245.146
74.52.245.146
66.249.65.43
66.249.65.43
66.249.65.43
66.249.65.12
66.249.65.12
66.249.65.12
66.249.65.12
66.249.65.12
66.249.65.12
66.249.65.12
65.55.207.25
65.55.207.25
66.249.65.12
66.249.65.12
66.249.65.12
66.249.65.12
66.249.65.12
66.249.65.12
66.249.65.12
66.249.65.12
65.55.207.94
65.55.207.94
66.249.65.12
65.55.207.71
66.249.65.12
66.249.65.12
66.249.65.12
98.242.170.241
66.249.65.38
66.249.65.38
66.249.65.38

66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
65.55.207.126
65.55.207.126
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
82.34.9.20
82.34.9.20
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
65.55.106.155
65.55.106.155
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
65.55.207.77
65.55.207.77
66.249.65.38
67.218.116.165
66.249.65.38
208.80.193.28
66.249.65.38
66.249.65.38
66.249.65.38

66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
89.248.172.58
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38

```
66.249.65.38
67.195.112.35
67.195.112.35
67.195.112.35
67.195.112.35
67.195.112.35
67.195.112.35
67.195.112.35
67.195.112.35
67.195.112.35
67.195.112.35
67.195.112.35
67.195.112.35
67.195.112.35
67.195.112.35
67.195.112.35
67.195.112.35
66.249.65.38
65.55.207.50
65.55.207.50
65.55.207.50
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
65.55.215.75
65.55.215.75
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
66.249.65.38
```

In [58]:

```python
# now that we can get the IP address, let's count how often each appears

counts = {}   # we'll this with IP addreses (keys) and counts (values)

f = open('mini-access-log.txt')

for one_line in f:
    ip_address = one_line.split()[0]

    if ip_address in counts:      # have we seen this IP address before?
        counts[ip_address] += 1   # add 1 to the existing count
    else:
        counts[ip_address] = 1    # if not, add the new key-value pair

f.close()
```

In [59]:
```
print(counts)
```

```
{'67.218.116.165': 2, '66.249.71.65': 3, '65.55.106.183': 2, '66.249.65.12':
32, '65.55.106.131': 2, '65.55.106.186': 2, '74.52.245.146': 2, '66.249.65.4
3': 3, '65.55.207.25': 2, '65.55.207.94': 2, '65.55.207.71': 1, '98.242.170.
241': 1, '66.249.65.38': 100, '65.55.207.126': 2, '82.34.9.20': 2, '65.55.10
6.155': 2, '65.55.207.77': 2, '208.80.193.28': 1, '89.248.172.58': 22, '67.1
95.112.35': 16, '65.55.207.50': 3, '65.55.215.75': 2}
```

Next up

1. More practice reading from files
2. Writing to files
3. Using `with`

The skeleton/outline for reading from files:

```
f = open(FILENAME)

for one_line in f:   # one_line is a string
    WHATEVER_WE_WANT

f.close()
```

In [60]:
```
# I want to count the characters in a file

filename = 'passwd.txt'
total = 0

f = open(filename)

for one_line in f:
    total += len(one_line)  # len(one_line) returns the number of characters

f.close()

print(total)
```

```
9064
```

In [63]:
```
# you can also check characters -- how? By iterating over one_line, one char

filename = 'passwd.txt'
counts = {'a':0, 'e':0, 'i':0, 'o':0, 'u':0}  # let's count how many times e

f = open(filename)

for one_line in f:
    for one_character in one_line:
        if one_character in counts:     # is it a vowel?
            counts[one_character] += 1  # add 1 to its count
```

```
f.close()

print(counts)
```

```
{'a': 551, 'e': 701, 'i': 380, 'o': 268, 'u': 206}
```

Exercise: Sum numbers

The file `nums.txt` contains:

- Several lines, each containing either nothing or an integer
- There can be whitespace (spaces, tabs, etc.) before or after the integer
- Remember that `str.strip` removes the whitespace, returning a new string
- Remember that `int` turns a string into an integer

Sum the numbers in `nums.txt`.

In [64]:
```
!cat nums.txt
```

```
5
        10
        20
        3
                        20

25
```

In [67]:
```
filename = 'nums.txt'
total = 0

f = open(filename)

for one_line in f:
    if one_line.strip() != '':           # check that we have a non-empty st
        total += int(one_line.strip())   # run int on one_line.strip(), and

f.close()
print(total)
```

```
83
```

Exercise: Word count

Unix systems have traditionally come with a program called `wc` ("word count"). When you run `wc` on a file, it tells you how many lines, how many words, and how many characters are in the file.

I want you to write a program that does the same thing.

- Counting lines: Yes, including blank lines

- Counting words: Break words up on whitespace
- Counting characters: Everything

There is a file called `wcfile.txt` that is designed for this exercise.

In [69]:
```
!cat wcfile.txt
```

```
This is a test file.

It contains 28 words and 20 different words.

It also contains 165 characters.

It also contains 11 lines.

It is also self-referential.

Wow!
```

In [74]:
```
number_of_lines = 0
number_of_words = 0
number_of_chars = 0

filename = 'wcfile.txt'
f = open(filename)

for one_line in f:
    number_of_lines += 1   # each iteration is 1 line, so just add 1
    number_of_chars += len(one_line)
    number_of_words += len(one_line.split())  # break the line into a list c

f.close()

print(f'lines = {number_of_lines}')
print(f'words = {number_of_words}')
print(f'chars = {number_of_chars}')
```

```
lines = 11
words = 28
chars = 165
```

Writing to files

Writing to files is similar to reading from files:

1. We still use `open` to open a file, but we have to add a second argument, `w` (for writing).
2. We don't read from the file (i.e., we cannot), but we can invoke the `write` method, which writes a string to the file.
3. Remember that `write` does *not* add a newline to the end of what you write.
4. You must be sure to close the file! Otherwise, you don't know when (or sometimes if) your content will be written.

In [75]:
```
f = open('myfile.txt', 'w')   # when we use 'w', this means: I'm OK with rep

f.write('abcde\n')
f.write('fghijklmn\n')

f.close()
```

In [76]:
```
!cat myfile.txt
```

```
abcde
fghijklmn
```

Exercise: Dict to config

1. Define a small dict with several key-value pairs
2. Write the key-value pairs to a file -- what I call a config file, with one pair per line, and each key separated from its value with `=`.
3. When you're done, the number of lines in the file should equal the number of key-value pairs in your dict.

In [78]:
```
d = {'a':10, 'b':20, 'c':30}

f = open('myconfig.txt', 'w')

for key, value in d.items():   # get each key-value pair
    f.write(f'{key}={value}\n')

f.close()
```

In [79]:
```
!cat myconfig.txt
```

```
a=10
b=20
c=30
```

Isn't it annoying to `close`?

We invoke `close` to ensure that if there are issues with writing to the buffer, they'll come to the forefront right away.

If everyone (pretty much) wants to `close`, then why isn't it automatic?

The answer is, Python can't know when you want to close your file.

But it does provide a mechanism that is good in general, and helps to remove invocations of `close`.

I'm talking about `with`.

In [81]:
```
# f = open('myfile.txt', 'w')

with open('myfile.txt', 'w') as f:   # opening the file for writing
    f.write('abcde!!!\n')               # writing to the file
    f.write('fghijklmn!!!\n')           # writing more to the file

# no close is needed here -- the end of the "with" block runs it automatical
```

In [82]:
```
!cat myfile.txt
```

```
abcde!!!
fghijklmn!!!
```

Exercise: Use `with` on the dict-to-file program.

Rewrite that program such that it uses the modern `with` construct.

In [ ]:
```
d = {'a':10, 'b':20, 'c':30}

with open('myconfig.txt', 'w') as f:
    for key, value in d.items():   # get each key-value pair
        f.write(f'{key}={value}\n')
```

Next time: Functions

In [ ]:
```

```